

5-dars savol-javob

1. Training nima va Optimization nima?

Training — modelni data orqali o‘qitish jarayoni. Training so‘zining o‘zbekcha ma’nosi “mashq qildirish”, “o‘qitish”, “o‘rgatish” degani. Deep Learning va Computer Visionda training modelning rasmlar va labellar asosida o‘rganish jarayoni hisoblanadi.

Data — ma’lumot. Computer Visionda data ko‘pincha rasm yoki video bo‘ladi. Image Classificationda data rasm va uning labelidan iborat bo‘ladi.

Label — to‘g‘ri javob yoki class nomi. Masalan, mushuk rasmi uchun label `cat`, it rasmi uchun label `dog` bo‘ladi. Class — kategoriya yoki tur.

Model training vaqtida rasmni ko‘radi, prediction qiladi, prediction haqiqiy label bilan solishtiriladi, loss hisoblanadi va model o‘z weightlarini yangilaydi. Prediction — model taxmini. Loss — model qanchalik xato qilganini ko‘rsatadigan qiymat. Weight — model ichidagi o‘rganiladigan vazn qiymati.

Training jarayoni oddiy qilib shunday tushuniladi:

Rasm modelga beriladi

↓

Model prediction qiladi

↓

Prediction haqiqiy label bilan solishtiriladi

↓

Loss hisoblanadi

↓

Xato orqaga tarqatiladi

↓

Weightlar yangilanadi

↓

Model keyingi safar yaxshiroq prediction qilishga harakat qiladi

Masalan, cat vs dog classification modeli bor. Modelga it rasmi berildi. Model quyidagicha prediction chiqardi:

Dog: 0.30

Cat: 0.65

Bird: 0.05

Haqiqiy javob Dog, lekin model eng katta ehtimollikni Cat classiga bergan. Bu xato hisoblanadi. Training jarayonida shu xato loss orqali o'lchanadi va model keyingi safar itni yaxshiroq tanishi uchun weightlar yangilanadi.

Optimization — model xatosini kamaytirish uchun model parameterlarini yangilash jarayoni. Optimization so'zining o'zbekcha ma'nosi "yaxshilash", "optimallashtirish", "eng yaxshi holatga olib kelish" degani.

Parameter — model ichidagi o'rganiladigan qiymat. Deep Learning modelida weight va bias parameter hisoblanadi. Bias — model hisoblashiga qo'shimcha moslashuv beradigan qiymat.

Optimizationning asosiy maqsadi:

Lossni kamaytirish

↓

Model predictionini yaxshilash



Accuracy ni oshirish

Accuracy — model nechta javobni to‘g‘ri topganini ko‘rsatadigan metric. Metric — model natijasini o‘lchaydigan ko‘rsatkich.

Training va Optimization bir-biridan farq qiladi, lekin ular birga ishlaydi. Training umumiy o‘qitish jarayoni bo‘lsa, optimization shu training ichida model xatosini kamaytirish uchun weightlarni yangilash jarayonidir.

Oddiy hayotiy misolda o‘quvchi testga tayyorlanyapti. O‘quvchi masala ishlaydi, xato qiladi, xatosini ko‘radi, keyingi safar xatosini kamaytirishga harakat qiladi. Bu trainingga o‘xshaydi. Xatoni ko‘rib, qaysi usul bilan yaxshilanishini tanlash esa optimizationga o‘xshaydi.

Deep Learningda bu quyidagicha bo‘ladi:

Training = modelning ko‘p marta mashq qilishi
Optimization = har bir xatodan keyin modelni yaxshilash

Training bo‘lmasa, model o‘rganmaydi. Optimization bo‘lmasa, model xatosini kamaytira olmaydi. Shu sababli ikkalasi model o‘rganishida birga ishlaydi.

2. Forward Pass va Backpropagation o‘rtasidagi farq nima?

Forward Pass va Backpropagation training jarayonidagi ikki muhim yo‘nalish hisoblanadi. Ularning biri modeldan prediction olish uchun

ishlaydi, ikkinchisi esa xatoni orqaga tarqatib, modelni tuzatish uchun ishlaydi.

Forward Pass — input ma'lumot model ichidan oldinga qarab o'tib, output yoki prediction hosil qiladigan jarayon. Forward — oldinga, pass — o'tish degani.

Forward Pass umumiy ko'rinishi:

Input → Model → Prediction

Image Classificationda:

Image → CNN Model → Predicted Class

CNN — Convolutional Neural Network. O'zbekcha ma'nosi "konvolyutsion neyron tarmoq". CNN rasm ichidagi qirra, chiziq, rang o'zgarishi, shakl va obyekt qismlarini ajratishda ishlatiladi.

Forward Pass vaqtida rasm modelga kiradi. Model rasmni qatlamlar orqali qayta ishlaydi. Oxirida classlar bo'yicha score yoki probability chiqadi.

Score — modelning har bir class uchun chiqargan balli. Probability — ehtimollik.

Masalan:

Cat: 0.10

Dog: 0.85

Bird: 0.05

Bu model rasmni 85% ehtimol bilan Dog deb topganini bildiradi.

Forward Pass vaqtida model hali o‘rganmaydi. Weightlar yangilanmaydi. Model faqat hozirgi weightlari asosida prediction qiladi.

Backpropagation — lossdan kelgan xatoni model ichida orqaga tarqatib, gradientlarni hisoblash jarayoni. Backpropagation so‘zining o‘zbekcha izohi “xatoni orqaga tarqatish” degani. Back — orqaga, propagation — tarqalish.

Backpropagation umumiy ko‘rinishi:

Loss → Output Layer → Hidden Layers → Earlier Layers

Output Layer — model natijasi chiqadigan qatlam.

Hidden Layers — model ichidagi yashirin qatlamlar. Feature extraction va qayta ishlash shu qatlamlarda sodir bo‘ladi.

Gradient — lossni kamaytirish uchun weight qaysi yo‘nalishda o‘zgarishi kerakligini ko‘rsatadigan qiymat. Gradient modelga “qaysi tomonga o‘zgarsa xato kamayadi” degan signal beradi.

Forward Pass va Backpropagation farqini jadval orqali tushunish mumkin:

Taqqoslash	Forward Pass	Backpropagation
Yo‘nalish	Oldinga qarab yuradi	Orqaga qarab yuradi
Boshlanishi	Inputdan boshlanadi	Lossdan boshlanadi
Vazifasi	Prediction chiqarish	Gradient hisoblash
Weightlar	O‘zgarmaydi	O‘zgarishga tayyorlanadi
Natija	Output / prediction	Gradientlar
Trainingdagi roli	Model javob beradi	Model xatosini tuzatishga yordam beradi

Forward Pass misoli:

Rasm: dog
Model output:
Dog: 0.30
Cat: 0.65
Bird: 0.05

Bu forward pass natijasi. Model prediction qildi, lekin xato qildi.

Keyin loss hisoblanadi. Loss katta bo'lsa, backpropagation ishga tushadi. Backpropagation model ichidagi weightlar uchun gradient hisoblaydi. Keyin optimizer shu gradientlar yordamida weightlarni yangilaydi.

Optimizer — gradientlardan foydalanib weightlarni yangilaydigan algoritm.

Forward Pass va Backpropagation o'rtasidagi asosiy farq:

Forward Pass = javob olish

Backpropagation = xatodan o'rganish uchun gradient hisoblash

Training step to'liq ko'rinishda shunday bo'ladi:

Forward Pass
↓
Loss hisoblash
↓
Backpropagation
↓
Optimizer Step

Optimizer Step — weightlarni yangilash qadami.

Forward Pass bo‘lmasa, prediction bo‘lmaydi. Prediction bo‘lmasa, loss hisoblab bo‘lmaydi. Loss bo‘lmasa, backpropagation bo‘lmaydi. Backpropagation bo‘lmasa, model qaysi weightni qanday tuzatishni bilmaydi.

3. Loss nimani o‘lchaydi?

Loss — model predictioni haqiqiy javobdan qanchalik uzoq ekanini o‘lchaydi. Loss model xatosini son ko‘rinishida beradi. Loss katta bo‘lsa, model predictioni yomonroq. Loss kichik bo‘lsa, model predictioni yaxshiroq.

Loss Function — lossni hisoblaydigan funksiya. Function — input olib, output qaytaradigan matematik qoida. Classification vazifalarida Cross-Entropy Loss ko‘p ishlatiladi.

Cross-Entropy Loss — model haqiqiy classga qanchalik probability berganiga qarab xatoni hisoblaydigan loss function.

Loss oddiy qilib quyidagicha tushuniladi:

Model prediction qildi

↓

Prediction haqiqiy label bilan solishtirildi

↓

Farq hisoblandi

↓

Shu farq loss bo‘ldi

Masalan, haqiqiy class Dog. Model quyidagicha prediction berdi:

Dog: 0.30
Cat: 0.65
Bird: 0.05

Bu yerda model Dog classiga 0.30 probability berdi, lekin Cat classiga 0.65 probability berdi. Haqiqiy javob Dog bo'lgani uchun model xato qildi.

Loss shu xatoni o'lchaydi.

Lossni tushunish uchun uch xil oddiy yondashuv bor.

1. Wrong-class confidence

Wrong-class confidence — noto'g'ri classga berilgan eng katta ishonch. Wrong class — noto'g'ri class. Confidence — modelning ishonch darajasi.

Yuqoridagi misolda:

True class = Dog

Prediction:

Dog: 0.30
Cat: 0.65
Bird: 0.05

Noto'g'ri classlar Cat va Bird. Ularning probabilitylari 0.65 va 0.05.

$\max(0.65, 0.05) = 0.65$

Bu model noto'g'ri classga 65% ishonch berganini bildiradi. Bu yomon holat.

2. Simple true-label error

Simple true-label error — haqiqiy classga berilgan probability asosida oddiy xato hisoblash.

Formula:

$$1 - P(\text{true class})$$

$P(\text{true class})$ — haqiqiy class probabilitysi.

Agar true class Dog bo'lsa va model Dogga 0.30 bergan bo'lsa:

$$1 - 0.30 = 0.70$$

Bu oddiy xato qiymati 0.70 bo'ladi.

3. Cross-Entropy Loss

Cross-Entropy Loss real trainingda ko'p ishlatiladi.

Formula:

$$\text{Loss} = -\log(P(\text{true class}))$$

Log — logarifm. Logarifm ehtimollik kichik bo'lsa lossni katta qiladi, ehtimollik yuqori bo'lsa lossni kichik qiladi.

Agar $P(\text{true class}) = 0.30$ bo'lsa:

$$\text{Loss} = -\log(0.30)$$

$$\text{Loss} \approx 1.20$$

Cross-Entropy Lossning asosiy mantiqi:

Haqiqiy class probabilitysi yuqori bo'lsa → loss kichik

Haqiqiy class probabilitysi past bo'lsa → loss katta

Misollar:

$P(\text{true class}) = 0.90 \rightarrow \text{loss kichik}$

$P(\text{true class}) = 0.50 \rightarrow \text{loss o'rtacha}$

$P(\text{true class}) = 0.10 \rightarrow \text{loss katta}$

$P(\text{true class}) = 0.01 \rightarrow \text{loss juda katta}$

Loss nima uchun kerak?

Loss modelga xatosini bildiradi. Agar loss bo'lmasa, model qanchalik noto'g'ri prediction qilganini bilmaydi. Loss trainingdagi asosiy signal hisoblanadi. Optimizer shu lossni kamaytirish uchun weightlarni yangilaydi.

Loss training davomida kuzatiladi. Agar loss pasayib borsa, model o'rganayotgan bo'ladi. Agar loss pasaymasa, data, model, optimizer, learning rate yoki preprocessingda muammo bo'lishi mumkin.

Loss curve — lossning vaqt bo'yicha o'zgarishini ko'rsatadigan grafik. Curve — egri chiziq. Trainingda loss curve odatda pastga qarab borishi kerak.

Qisqa javob:

Loss = model predictioni haqiqiy javobdan qanchalik uzoq ekanini o'lchaydi

4. Optimizer va Learning Rate qanday rol o'ynaydi?

Optimizer va Learning Rate model weightlarini yangilashda asosiy rol o'ynaydi. Backpropagation gradientlarni hisoblaydi, optimizer esa shu gradientlardan foydalanib weightlarni yangilaydi. Learning Rate esa yangilash qadamining kattaligini belgilaydi.

Optimizer nima rol o'ynaydi?

Optimizer — model weightlarini loss kamayadigan tomonga yangilaydigan algoritm. Optimizer so'zining o'zbekcha izohi “optimallashtiruvchi algoritm” yoki “modelni yaxshilovchi algoritm”.

Optimizer ishlash jarayoni:

Gradientlar hisoblanadi

↓

Optimizer gradientlarni oladi

↓

Weightlar yangilanadi

↓

Model keyingi safar yaxshiroq prediction qilishga harakat qiladi

Weight update oddiy formula bilan tushuniladi:

$$\text{New weight} = \text{Old weight} - \text{Learning Rate} \times \text{Gradient}$$

New weight — yangi weight.

Old weight — eski weight.

Learning Rate — qadam kattaligi.

Gradient — lossni kamaytirish yo'nalishini ko'rsatuvchi qiymat.

Optimizer modelni xatoni kamaytiradigan yoʻnalishda harakatlantiradi. Masalan, model dog rasmiga cat deb javob berdi. Loss katta boʻldi. Backpropagation gradientlarni hisoblaydi. Optimizer weightlarni yangilaydi. Keyingi safar model dogga koʻproq probability berishga harakat qiladi.

Mashhur optimizerlar:

SGD

Adam

AdamW

RMSprop

SGD — Stochastic Gradient Descent. Stochastic — kichik batchlar yoki tasodifiy namunalar asosida ishlash. Gradient Descent — gradient boʻyicha lossni kamaytirish.

Adam — Adaptive Moment Estimation. Adam gradientlarning yoʻnalishi va oʻzgarish tezligini hisobga olib, weightlarni moslashuvchan yangilaydi. Adaptive — moslashuvchan.

AdamW — Adamning weight decay bilan ishlaydigan varianti. Weight decay — weightlarni juda kattalashib ketishdan saqlaydigan regularization usuli.

RMSprop — gradient oʻzgarishlarini hisobga olib ishlaydigan optimizer.

Regularization — modelning training data'ni haddan tashqari yodlab olishining oldini olish usullari.

Learning Rate nima rol o'ynaydi?

Learning Rate — optimizer har bir qadamda weightlarni qanchalik katta o'zgartirishini belgilaydi. Learning Rate qisqacha η deb yoziladi.

Learning Rate juda kichik bo'lsa:

model juda sekin o'rganadi
loss juda sekin kamayadi
training uzoq vaqt oladi

Learning Rate juda katta bo'lsa:

model minimumdan sakrab o'tadi
loss tebranadi
training diverge bo'lishi mumkin

Diverge — training beqaror bo'lib, loss kamaymasdan ketishi yoki juda katta bo'lib ketishi.

Learning Rate to'g'ri tanlansa:

loss barqaror kamayadi
model tezroq o'rganadi
training yaxshi ketadi

Learning Rate qadam uzunligiga o'xshaydi. Tog'dan pastga tushishda juda kichik qadam bilan yurilsa, pastga tushish juda sekin bo'ladi. Juda katta qadam bilan yurilsa, muvozanat yo'qoladi yoki kerakli joydan o'tib ketiladi. Mos qadam bilan yurilsa, pastga tez va barqaror tushiladi.

Deep Learningda ham loss minimumiga tushish uchun learning rate mos bo'lishi kerak.

Learning Rate misollari:

0.1 → katta bo'lishi mumkin
0.01 → o'rtacha
0.001 → ko'p ishlatiladigan qiymat
0.0001 → kichikroq qiymat

Lekin eng yaxshi learning rate model, dataset, optimizer va batch sizga bog'liq bo'ladi.

PyTorchda optimizer va learning rate misoli:

```
optimizer = torch.optim.Adam(model.parameters()),  
lr=0.001)
```

Bu yerda:

Adam → optimizer

model.parameters() → model weight va biaslari

lr=0.001 → learning rate

Optimizer va Learning Rate farqi:

Tushuncha	Vazifasi
Optimizer	Weightlarni qanday yangilashni boshqaradi
Learning Rate	Weightlar qanchalik katta qadam bilan yangilanishini belgilaydi

Qisqa javob:

Optimizer = weightlarni yangilaydi

Learning Rate = yangilash qadamining kattaligini

belgilaydi

5. Epoch va Batch nima?

Epoch va Batch training jarayonining qanday takrorlanishini tushuntiradigan muhim tushunchalar hisoblanadi.

Batch — datasetning kichik qismi. Model odatda butun datasetni birdaniga koʻrmaydi. Dataset batchlarga boʻlinadi va model har bir batchni alohida qayta ishlaydi.

Epoch — model butun training datasetni bir marta toʻliq koʻrib chiqishi.

Batch nima?

Batch — model bir training stepda koʻradigan maʼlumotlar guruhi. Image Classificationda bitta batch ichida bir nechta rasm va ularning labellari boʻladi.

Masalan:

`Batch size = 32`

Bu model bir training stepda 32 ta rasmni koʻradi degani.

Batch size — bitta batch ichidagi samplelar soni. Sample — bitta maʼlumot birligi. Image Classificationda bitta sample bitta rasm va uning labelidan iborat boʻladi.

Batch ishlatish sabablari:

xotirani tejash
trainingni tezlashtirish

GPUdan samarali foydalanish
gradientlarni hisoblashni qulay qilish

GPU — Graphics Processing Unit, grafik protsessor. Deep Learningda trainingni tezlashtirish uchun ishlatiladi.

Agar datasetda 10 000 ta rasm bo'lsa, hammasini birdan modelga berish xotiraga og'ir bo'ladi. Shu sababli rasmlar batchlarga bo'linadi.

Epoch nima?

Epoch — model butun training datasetni bir marta to'liq ko'rib chiqishi.

Masalan:

Dataset size = 1000 ta rasm

Batch size = 100

Bitta epoch nechta batchdan iborat bo'ladi?

$1000 / 100 = 10$ batch

Ya'ni 1 epoch = 10 batch.

Agar training 5 epoch bo'lsa:

$10 \text{ batch} \times 5 \text{ epoch} = 50 \text{ training step}$

Training step — bitta batch bo'yicha forward pass, loss, backpropagation va optimizer step bajariladigan jarayon.

Epochni kitob o'qish misolida tushunish mumkin. Kitobdagi hamma sahifalarni bir marta o'qib chiqish — 1 epoch. Har safar bir nechta sahifa o'qish — batch. Barcha sahifalar tugasa, 1 epoch tugaydi.

Batch va Epoch farqi:

Tushuncha	Ma'nosi
Batch	Datasetning kichik qismi
Batch size	Bitta batch ichidagi samplelar soni
Epoch	Butun datasetni bir marta to'liq ko'rish
Iteration	Bitta batch bo'yicha training step

Iteration — bitta batch ustida bajarilgan training qadami.

Misol:

Dataset = 800 ta rasm

Batch size = 16

Bitta epoch nechta iterationdan iborat?

$800 / 16 = 50$ iteration

Agar epoch = 10 bo'lsa:

$50 \times 10 = 500$ iteration

PyTorch training loop misoli:

```
for epoch in range(num_epochs):
    for images, labels in train_loader:
        outputs = model(images)
        loss = criterion(outputs, labels)
        optimizer.zero_grad()
        loss.backward()
```

`optimizer.step()`

Tashqi loop epoch bo'yicha aylanadi. Ichki loop batchlar bo'yicha aylanadi.

Batch katta bo'lsa:

training barqarorroq bo'lishi mumkin
xotira ko'proq kerak bo'ladi

Batch kichik bo'lsa:

xotira kamroq kerak bo'ladi
gradient shovqinliroq bo'ladi
ba'zan generalization yaxshi bo'ladi

Generalization — modelning yangi data'da yaxshi ishlashi.

Qisqa javob:

Batch = datasetning kichik qismi
Epoch = butun datasetdan bir marta to'liq o'tish

6. Model training tugagandan keyin next step nima bo'ladi?

Model training tugagandan keyin keyingi bosqich modelni real hayotda ishlatishga tayyorlash bo'ladi. Training tugashi model productga tayyor bo'ldi degani emas. Model avval baholanadi, saqlanadi, inference uchun tayyorlanadi, deployment qilinadi va monitoring qilinadi.

Trainingdan keyingi asosiy bosqichlar:

1. Evaluation
2. Best modelni save qilish
3. Inference pipeline tayyorlash
4. Deployment optimization
5. API yoki App yaratish
6. Deployment
7. Monitoring
8. Kerak bo'lsa retraining

1. Evaluation

Evaluation — modelni baholash. Training tugagandan keyin model test datasetda tekshiriladi. Test dataset trainingda ishlatilmagan bo'lishi kerak. Shunda model yangi data'da qanday ishlashi adolatli baholanadi.

Evaluation metriclar:

Accuracy

Precision

Recall

F1-score

Confusion Matrix

Accuracy — umumiy to'g'ri javoblar ulushi.

Precision — model bir class deb aytganlar ichida nechta to'g'ri ekanini bildiradi.

Recall — haqiqiy classlardan nechta model topganini bildiradi.

F1-score — precision va recall o'rtasidagi muvozanat.

Confusion Matrix — model qaysi classni qaysi class bilan adashtirganini ko‘rsatadigan jadval.

2. Best modelni save qilish

Training davomida eng yaxshi validation natija bergan model saqlanadi. Validation — training paytida modelni tekshirish uchun ishlatiladigan alohida data.

PyTorchda model odatda .pth formatda saqlanadi:

```
torch.save(model.state_dict(), "best_model.pth")
```

best_model.pth — eng yaxshi model weightlari saqlangan fayl.

3. Inference pipeline tayyorlash

Inference pipeline — yangi rasm kelganda model prediction berishi uchun kerakli jarayon.

Pipeline — ketma-ket bajariladigan bosqichlar.

Inference pipeline:

New Image

↓

Preprocess

↓

Load Model

↓

model.eval()

↓

Forward Pass

↓

Softmax

↓

Predicted Class

Preprocess — rasmni modelga moslash. Trainingda qanday preprocessing ishlatilgan bo'lsa, inference paytida ham shunga mos bo'lishi kerak.

`model.eval()` — modelni inference modega o'tkazadi.

Softmax — class scorelarni probabilityga aylantiradi.

4. Deployment Optimization

Model productga chiqarilishidan oldin yengillashtirilishi va tezlashtirilishi mumkin.

Deployment Optimization usullari:

Quantization

ONNX export

TensorRT

Smaller backbone

Image size kamaytirish

Quantization — float32 qiymatlarni int8 kabi kichik formatga o'tkazish.

ONNX — modelni turli framework va runtime orasida ishlatishga yordam beradigan universal format.

TensorRT — NVIDIA GPU uchun optimized inference engine.

Backbone — feature extraction qiladigan asosiy model qismi.

5. API yoki App yaratish

Model foydalanuvchi ishlata olishi uchun API yoki app kerak bo‘ladi.

API — Application Programming Interface. Boshqa dasturlar modelga rasm yuborib, natija olishi uchun interfeys.

App — foydalanuvchi ishlaydigan ilova. Masalan, Gradio yoki Streamlit bilan oddiy image upload app qilish mumkin.

Tools:

FastAPI

Flask

Gradio

Streamlit

FastAPI va Flask backend API qilish uchun ishlatiladi. Backend — requestni qabul qilib, modelga yuboradigan server qismi.

Gradio va Streamlit tez web interface qilish uchun qulay. Interface — foydalanuvchi ko‘radigan ekran.

6. Deployment

Deployment — modelni real tizimga joylash. Model local PC, server, web app, mobile app yoki edge devicega chiqarilishi mumkin.

Deployment joylari:

Local PC

Server

Cloud

Mobile app

Edge device

Cloud — internetdagi server muhiti. Edge device — telefon, kamera, Raspberry Pi, Jetson kabi kichik qurilma.

7. Monitoring

Deploymentdan keyin model kuzatiladi. Monitoring — productda ishlayotgan modelning natijasi, tezligi va xatolarini kuzatish.

Monitoringda quyidagilar tekshiriladi:

Accuracy

Latency

Failures

Data drift

Prediction distribution

Latency — bitta requestga javob qaytarish vaqti.

Failures — xatolar, crash, timeout.

Data drift — real data training data'dan farq qilib ketishi.

Prediction distribution — model qaysi classlarni qanchalik ko'p prediction qilayotganini ko'rsatadi.

8. Retraining

Agar model real data'da yomonlashsa, yangi data yig'ilib model qayta train qilinadi.

Retraining — modelni qayta o'qitish.

Retraining jarayoni:

Real xatolar yig'iladi

↓

Yangi dataset tayyorlanadi

↓
Model qayta train qilinadi
↓
Evaluation qilinadi
↓
Yangi model deploy qilinadi

Trainingdan keyingi umumiy flow:

Training tugadi
↓
Evaluation
↓
Save best model
↓
Inference script
↓
API/App
↓
Deployment
↓
Monitoring
↓
Retraining

Qisqa javob:

Training tugagandan keyingi next step – modelni
evaluation qilish, saqlash, inference uchun
tayyorlash, deploy qilish va monitoring qilish

7. O‘tilgan barcha CV va Deep Learningdagi kutubxonalar ro‘yxati va ularning vazifalari

Computer Vision va Deep Learning projectlarida ko‘plab kutubxona va tool ishlatiladi. Kutubxona — tayyor funksiyalar to‘plami. Tool — ishni bajarishga yordam beradigan vosita.

Quyida o‘tilgan va shu mavzularda ishlatiladigan asosiy kutubxona hamda toollar ro‘yxati beriladi.

1. PyTorch

PyTorch — Deep Learning model yaratish, train qilish va inference qilish uchun ishlatiladigan framework.

Framework — model qurish va ishlatish uchun tayyor vositalar to‘plami.

PyTorch vazifalari:

- tensor bilan ishlash
- neural network yaratish
- loss hisoblash
- backpropagation bajarish
- optimizer ishlatish
- model save/load qilish
- GPUda training qilish

Asosiy qismlari:

- torch
- torch.nn
- torch.optim

`torch.utils.data`

`torch` — tensor va asosiy hisoblashlar uchun.

`torch.nn` — neural network layerlari uchun.

`torch.optim` — optimizerlar uchun.

`torch.utils.data` — Dataset va DataLoader uchun.

2. Torchvision

Torchvision — PyTorch ekosistimidagi Computer Vision kutubxonasi.
U datasetlar, image transformlar va tayyor modellarga ega.

Torchvision vazifalari:

- CIFAR10, MNIST kabi datasetlarni yuklash
- image transform qilish
- pretrained model ishlatish
- image classification uchun tayyor pipeline tuzish

Asosiy qismlari:

- `torchvision.datasets`
- `torchvision.transforms`
- `torchvision.models`

`torchvision.datasets` — tayyor datasetlar.

`torchvision.transforms` — resize, normalize, crop, tensor conversion kabi preprocessing amallari.

`torchvision.models` — ResNet, MobileNet, EfficientNet kabi tayyor modellar.

3. OpenCV

OpenCV — Computer Vision va image processing uchun juda mashhur kutubxona.

Image processing — rasm ustida ishlov berish. Masalan, rasm o‘qish, o‘lchamini o‘zgartirish, rang formatini almashtirish, chizma chizish.

OpenCV odatda cv2 nomi bilan import qilinadi.

OpenCV vazifalari:

- rasm o‘qish
- video o‘qish
- frame ajratish
- resize qilish
- color conversion
- drawing
- image filtering
- camera bilan ishlash

Color conversion — rang formatini almashtirish. Masalan, BGRdan RGBga o‘tkazish.

BGR — Blue, Green, Red. OpenCV rasmni ko‘pincha BGR formatda o‘qiydi.

RGB — Red, Green, Blue. Ko‘p Deep Learning pipeline RGB format kutadi.

4. PIL / Pillow

PIL yoki Pillow — rasm fayllarini o‘qish, ochish va oddiy image processing qilish uchun ishlatiladi.

Pillow vazifalari:

image open qilish
image resize qilish
image crop qilish
image format o'zgartirish
PyTorch transformlar bilan ishlash

PIL ko'pincha `Image.open()` orqali ishlatiladi.

5. NumPy

NumPy — sonli hisoblashlar va array bilan ishlash uchun kutubxona.

Array — raqamlar jadvali yoki ko'p o'lchamli massiv.

NumPy vazifalari:

array yaratish
matematik hisoblash
pixel qiymatlari bilan ishlash
matrix operatsiyalar
statistik hisoblash

Computer Visionda rasm ko'pincha NumPy array ko'rinishida ham ishlatiladi.

6. Pandas

Pandas — jadval ma'lumotlar bilan ishlash uchun kutubxona. CV projectlarda metadata, label fayllari yoki CSV datasetlar bilan ishlashda foydali.

Pandas vazifalari:

CSV o'qish
jadval ko'rish

label ustunlarini boshqarish
dataset statistikasi olish
missing value tekshirish

Metadata — rasm haqida qo'shimcha ma'lumot. Masalan, filename, label, path, width, height.

7. Matplotlib

Matplotlib — grafik va rasm ko'rsatish uchun kutubxona.

Matplotlib vazifalari:

rasm ko'rsatish
loss curve chizish
accuracy curve chizish
bar chart chizish
training natijalarini ko'rsatish

Loss curve — training davomida loss qanday o'zgarishini ko'rsatadigan grafik.

Accuracy curve — accuracy qanday o'zgarishini ko'rsatadigan grafik.

8. Seaborn

Seaborn — statistik grafiklar chizish uchun kutubxona. Matplotlib ustiga qurilgan.

Seaborn vazifalari:

confusion matrix heatmap qilish
statistik plotlar chizish
dataset taqsimotini ko'rsatish

Heatmap — qiymatlarni ranglar orqali ko‘rsatadigan grafik.

9. Scikit-learn

Scikit-learn yoki sklearn — Machine Learning va evaluation metriclar uchun ishlatiladi.

CV projectlarda sklearn ko‘pincha evaluation uchun ishlatiladi.

Sklearn vazifalari:

```
train_test_split  
accuracy_score  
precision_score  
recall_score  
f1_score  
confusion_matrix  
classification_report
```

`train_test_split` — datasetni train va testga ajratish.

`classification_report` — precision, recall, F1-score kabi metriclarni chiqarish.

10. Albumentations

Albumentations — image augmentation uchun kuchli kutubxona.

Augmentation — rasmni turli ko‘rinishga keltirib datasetni boyitish.

Albumentations vazifalari:

```
rotate  
flip  
crop  
blur
```

brightness change
contrast change
noise qo'shish

Noise — rasmga qo'shiladigan tasodifiy shovqin.

Albumentations real Computer Vision projectlarda tez va kuchli augmentation qilish uchun ishlatiladi.

11. Scikit-image

Scikit-image — image processing uchun Python kutubxonasi.

Vazifalari:

image filtering
edge detection
segmentation yordamchi amallari
color conversion
image measurement

Edge detection — rasm ichidagi qirralarni aniqlash.

Segmentation — rasmni hududlarga ajratish.

12. FastAPI

FastAPI — Python'da API yaratish uchun zamonaviy framework.

Deploymentda FastAPI modelni serverga chiqarishda ishlatiladi.

FastAPI vazifalari:

image upload endpoint yaratish
request qabul qilish
model inference bajarish

prediction response qaytarish

Endpoint — API ichidagi ma'lum URL manzil. Masalan, /predict.

Request — foydalanuvchi yuborgan so'rov.

Response — server qaytargan javob.

13. Flask

Flask — Python'da web app yoki API yaratish uchun yengil framework.

Flask vazifalari:

- oddiy API yaratish
- modelni web orqali ishlatish
- image upload qabul qilish
- prediction qaytarish

Flask FastAPIga qaraganda soddaroq, lekin ko'p projectlarda ishlatiladi.

14. Gradio

Gradio — Machine Learning modeli uchun tez va oddiy web interface yaratishga yordam beradigan tool.

Gradio vazifalari:

- rasm yuklash oynasi yaratish
- model predictionini ko'rsatish
- demo app qilish
- local inference app yaratish

Gradio beginner projectlar uchun juda qulay, chunki kam kod bilan model demo qilish mumkin.

15. Streamlit

Streamlit — Python orqali interaktiv web app yaratish uchun kutubxona.

Streamlit vazifalari:

image upload app qilish
prediction natijasini ko'rsatish
dashboard yaratish
model demo qilish

Dashboard — ma'lumot va natijalarni ko'rsatadigan panel.

16. ONNX

ONNX — Open Neural Network Exchange. Modelni turli framework va runtime orasida ishlatish uchun universal format.

ONNX vazifalari:

PyTorch modelni universal formatga o'tkazish
deploymentni osonlashtirish
ONNX Runtime orqali inference qilish
portabilityni oshirish

Portability — modelni turli muhitlarda ishlata olish qulayligi.

17. ONNX Runtime

ONNX Runtime — ONNX formatdagi modelni ishga tushirish uchun runtime.

Runtime — model bajariladigan muhit.

ONNX Runtime vazifalari:

ONNX modelni tez inference qilish
server yoki edge device'da ishlatish
frameworkga kamroq bog'liq bo'lish

18. TensorRT

TensorRT — NVIDIA GPU uchun optimized inference engine.

TensorRT vazifalari:

inference tezligini oshirish
latency kamaytirish
GPUda modelni optimallashtirish
real-time systemlarda ishlash

Real-time — real vaqt rejimida ishlash.

19. Docker

Docker — projectni container ichida bir xil environment bilan ishga tushirish vositasi.

Container — kod, kutubxonalar, sozlamalar birga joylashgan paket.

Docker vazifalari:

environment muammolarini kamaytirish
serverda projectni bir xil ishga tushirish
deploymentni barqaror qilish
requirements va versiyalarni boshqarish

Environment — dastur ishlaydigan muhit. Masalan, Python versiyasi, kutubxona versiyalari, CUDA versiyasi.

20. LabelImg

LabelImg — image annotation uchun tool. Object Detectionda bounding box chizish uchun ishlatiladi.

Annotation — rasmga label yoki belgi qo'yish.

Bounding box — obyekt atrofidagi to'rtburchak ramka.

21. CVAT

CVAT — Computer Vision Annotation Tool. Rasm va videolarni annotation qilish uchun kuchli tool.

CVAT vazifalari:

bounding box chizish
segmentation mask qilish
video annotation
dataset label qilish

Mask — segmentationda obyektни pixel darajasida ajratish.

22. Kaggle

Kaggle — datasetlar, notebooklar va Machine Learning competitionlar platformasi.

Kaggle vazifalari:

CV dataset topish
project dataset yuklab olish
notebookda model train qilish

competitionlarda qatnashish

Competition — musobaqa.

23. Hugging Face Datasets

Hugging Face Datasets — turli datasetlarni yuklash va ishlatish uchun platforma.

Vazifalari:

image dataset topish
community datasetlardan foydalanish
custom dataset hosting

Community dataset — foydalanuvchilar yoki tadqiqotchilar joylagan dataset.

24. TensorFlow Datasets

TensorFlow Datasets — Machine Learning uchun tayyor datasetlar katalogi.

Vazifalari:

tayyor dataset yuklash
TensorFlow ekotizimida dataset ishlatish
turli ML/CV datasetlar bilan tajriba qilish

Umumiy xulosa:

PyTorch → model yaratish va train qilish
Torchvision → CV dataset, transform, model
OpenCV → rasm/video processing

Pillow → rasm ochish va oddiy processing
NumPy → raqamli hisoblash
Pandas → jadval va label bilan ishlash
Matplotlib/Seaborn → grafik va visualization
Sklearn → evaluation metrics
Albumentations → augmentation
FastAPI/Flask → API deployment
Gradio/Streamlit → demo app
ONNX/ONNX Runtime/TensorRT → deployment
optimization va inference
Docker → environment va deployment
LabelImg/CVAT → annotation
Kaggle/Hugging Face/TensorFlow Datasets → dataset
manbalari

8. O‘tilgan 3 ta phase’ni alohida-alohida architecture sifatida chizish

O‘tilgan 3 ta asosiy phase quyidagicha olinadi:

Phase 1: Data Preparation

Phase 2: Model Architecture

Phase 3: Training & Optimization

Har bir phase uchun alohida architecture chiziladi. Bu chizmalar darsdagi tayyor chizmalarni aynan ko‘chirmaydi, balki mazmunni boshqa ko‘rinishda tushuntiradi.

Phase 1: Data Preparation Architecture

Data Preparation — modelga beriladigan rasm datasetini tayyorlash bosqichi. Bu bosqichda rasm yig‘iladi, label tekshiriladi, folder structure tuziladi, rasm tozalanadi, train/validation/testga bo‘linadi va preprocessing pipeline tayyorlanadi.

Custom architecture:

Raw Image Sources



Image Collection



Label Checking



Folder Structure



Image Cleaning



Resize / Normalize Rules



Train / Validation / Test Split



Dataset Class



DataLoader



Ready Batches for Model

1. Raw Image Sources

Raw Image Sources — boshlang'ich rasm manbalari. Rasmlar internetdan, kamera orqali, Kaggle'dan, Hugging Face'dan yoki lokal papkadan olinishi mumkin.

Raw — xom, hali ishlov berilmagan.

2. Image Collection

Image Collection — rasm yig'ish. Har bir class uchun yetarli rasm yig'iladi.

Masalan:

cat → 1000 ta rasm

dog → 1000 ta rasm

Classlar soni juda noteng bo'lsa, class imbalance paydo bo'ladi.

Class imbalance — classlar orasida data soni noteng bo'lishi.

3. Label Checking

Label Checking — label to'g'riligini tekshirish. Masalan, cat papkasida dog rasmi bo'lmasligi kerak.

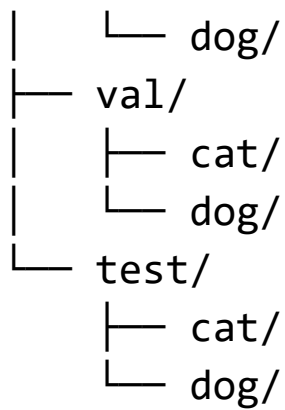
Noto'g'ri label modelni noto'g'ri o'rgatadi.

4. Folder Structure

Folder Structure — dataset papkalarini tartibli tuzish.

Masalan:

```
data/  
├─ train/  
│   └─ cat/
```



Bu structure modelga labelni papka nomidan olishga yordam beradi.

5. Image Cleaning

Image Cleaning — yomon rasmlarni tozalash.

Tozalanadigan holatlar:

buzilgan rasm
ochilmaydigan fayl
juda xira rasm
 noto'g'ri classdagi rasm
takroriy rasm

6. Resize / Normalize Rules

Resize — hamma rasmni bir xil o'lchamga keltirish.

Normalize — pixel qiymatlarini model uchun qulay scale'ga keltirish.

Masalan:

Resize: 224 × 224

Normalize: mean/std asosida

7. Train / Validation / Test Split

Dataset uch qismga bo'linadi:

Train → model o'rganadi

Validation → training paytida tekshiriladi

Test → yakuniy baholash

8. Dataset Class

Dataset Class — PyTorchda rasm va labelni o'qib beradigan class. Bu class `__getitem__` orqali bitta sample qaytaradi.

9. DataLoader

DataLoader — datasetdan batch tayyorlab modelga uzatadigan vosita.

DataLoader vazifalari:

batch qilish

shuffle qilish

parallel loading

modelga tensor berish

Shuffle — data tartibini aralashtirish.

10. Ready Batches for Model

Oxirida modelga tayyor batchlar chiqadi.

Batch ichida:

images

labels

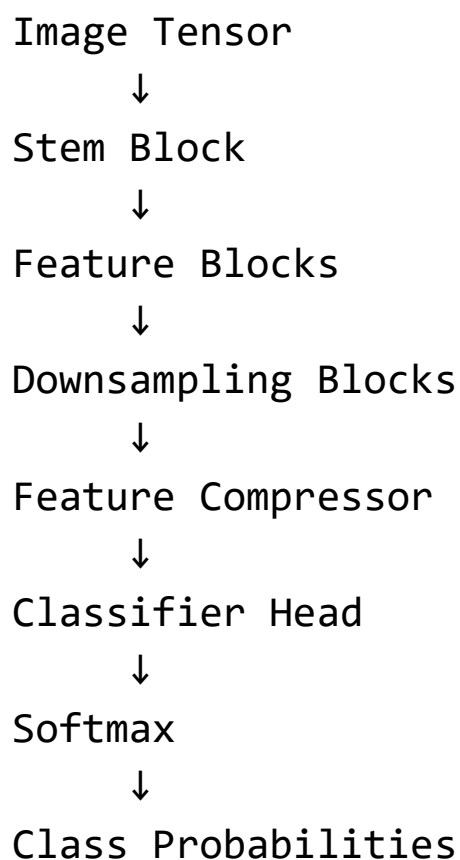
Data Preparation phase xulosasi:

Rasm yig'iladi → tozalanadi → label tekshiriladi → split qilinadi → transform qilinadi → batch bo'lib modelga tayyor bo'ladi

Phase 2: Model Architecture

Model Architecture — modelning ichki tuzilishini yaratish bosqichi. Bu bosqichda input rasm qaysi qatlamlardan o'tishi, featurelar qayerda ajratilishi va output qanday chiqishi belgilanadi.

Custom architecture:



1. Image Tensor

Image Tensor — rasmning modelga kiradigan raqamli ko‘rinishi.

Masalan:

```
[batch_size, channels, height, width]
```

PyTorchda rasm odatda shunday formatda bo‘ladi:

```
[32, 3, 224, 224]
```

Bu yerda:

32 → batch size

3 → RGB kanal

224 → height

224 → width

2. Stem Block

Stem Block — modelning boshlang‘ich feature extraction qismi. Stem — boshlanish qismi.

Bu qism rasm ichidan dastlabki oddiy featurelarni ajratadi:

qirra

chiziq

rang o‘zgarishi

oddiy shakl

3. Feature Blocks

Feature Blocks — rasm ichidan chuqurroq va murakkabroq belgilarni oʻrganadigan bloklar.

Masalan:

koʻz shakli
quloq shakli
gʻildirak
barg dogʻi
hayvon yuzi

CNNda bu bloklar convolutional layersdan iborat boʻladi.

4. Downsampling Blocks

Downsampling Blocks — feature map hajmini kichraytiradigan bloklar. Downsampling hisoblashni kamaytiradi va muhim featurelarni saqlab qoladi.

Pooling yoki stride convolution ishlatilishi mumkin.

Stride — filter rasm ustida nechta pixel qadam bilan harakatlanishini bildiradi.

5. Feature Compressor

Feature Compressor — featurelarni ixcham vector koʻrinishiga keltiradi.

Bu yerda Global Average Pooling ishlatilishi mumkin.

Global Average Pooling:

$$H \times W \times C \rightarrow 1 \times 1 \times C$$

H — height.

W — width.

C — channel.

6. Classifier Head

Classifier Head — yakuniy class scorelarni chiqaradigan qism.

Bu qismda Fully Connected Layer ishlatiladi.

Fully Connected Layer — har bir kiruvchi feature har bir chiqish neuroniga ulangan qatlam.

7. Softmax

Softmax — class scorelarni probabilityga aylantiradi.

Masalan:

Cat score = 3.2

Dog score = 1.1

Softmaxdan keyin:

Cat = 0.89

Dog = 0.11

8. Class Probabilities

Class Probabilities — har bir class uchun ehtimollik qiymatlari.

Model eng katta probabilityga ega classni prediction sifatida tanlaydi.

Model Architecture phase xulosasi:

Rasm tensor bo'lib kiradi → featurelar ajratiladi → featurelar ixchamlanadi → classifier class score chiqaradi → softmax probability beradi

Phase 3: Training & Optimization Architecture

Training & Optimization — modelni o'qitish, lossni kamaytirish va weightlarni yangilash bosqichi.

Custom architecture:

Batch from DataLoader



Forward Pass



Prediction Scores



Loss Function



Gradient Calculation



Optimizer Update



Validation Check



Save Best Model



Repeat for Next Epoch

1. Batch from DataLoader

DataLoader modelga batch beradi.

Batch ichida:

`images`

`labels`

Images — rasmlar.

Labels — haqiqiy javoblar.

2. Forward Pass

Forward Pass — images modeldan o'tib prediction scores hosil qiladi.

```
outputs = model(images)
```

3. Prediction Scores

Prediction Scores — model chiqargan class scorelar.

Masalan:

Cat: 1.2

Dog: 3.4

Bird: -0.5

Bu scorelar softmax orqali probabilityga aylanishi mumkin.

4. Loss Function

Loss Function prediction va label orasidagi xatoni hisoblaydi.

```
loss = criterion(outputs, labels)
```

Criterion — loss function nomi sifatida ishlatiladi.

Classificationda ko‘pincha CrossEntropyLoss ishlatiladi.

5. Gradient Calculation

Gradient Calculation — backpropagation orqali gradientlarni hisoblash.

```
loss.backward()
```

Gradientlar weightlar qaysi yo‘nalishda o‘zgarishi kerakligini bildiradi.

6. Optimizer Update

Optimizer weightlarni yangilaydi.

```
optimizer.step()
```

Oldingi gradientlarni tozalash uchun:

```
optimizer.zero_grad()
```

ishlatiladi.

7. Validation Check

Validation Check — har epochdan keyin model validation datasetda tekshiriladi.

Kuzatiladigan metriklar:

validation loss
validation accuracy
F1-score

8. Save Best Model

Agar validation natija oldingidan yaxshi bo'lsa, model saqlanadi.

`best_model.pth`

Bu keyin inference va deploymentda ishlatiladi.

9. Repeat for Next Epoch

Training bir nechta epoch davom etadi. Har epochda model butun training datasetni yana bir marta ko'radi.

Training & Optimization phase xulosasi:

Batch olinadi → model prediction qiladi → loss hisoblanadi → gradientlar topiladi → optimizer weightlarni yangilaydi → validation tekshiriladi → eng yaxshi model saqlanadi

3 ta phase umumiy birlashgan ko'rinish

PHASE 1: DATA PREPARATION

Raw Images → Clean Data → Split → Transform → DataLoader

PHASE 2: MODEL ARCHITECTURE

Image Tensor → Feature Extractor → Feature Compressor → Classifier → Probabilities

PHASE 3: TRAINING & OPTIMIZATION

Batch → Forward → Loss → Backward → Optimizer Step → Validation → Save Model

Bu 3 phase bir-birini to'ldiradi. Data Preparation bo'lmasa, modelga sifatli input bo'lmaydi. Model Architecture bo'lmasa, rasmni qayta ishlaydigan structure bo'lmaydi. Training & Optimization bo'lmasa, model o'rganmaydi.

Qisqa umumiy xulosa:

Phase 1 = data tayyorlanadi

Phase 2 = model tuziladi

Phase 3 = model o'qitiladi va yaxshilanadi